

# AI-Enabled Intelligent Defense for Link Flooding Attacks in Software Defined Networks

Qiang He , Associate Member, IEEE, Quanwei Li , Chuangchuang Zhang , Fuliang Li , Member, IEEE, Xingwei Wang , Chi Xu , Senior Member, IEEE, Ammar Hawbani , and Keping Yu , Senior Member, IEEE

**Abstract**—Link Flooding Attack (LFA) can disconnect target links or network areas to cause the disruption of legitimate services by flooding network traffic. The centralized control makes Software Defined Network (SDN) vulnerable to LFA attacks. Most of existing LFA defense solutions based on statistic analysis and machine learning have the limitations on obtaining sufficient samples, detection accuracy and mitigation effect. In this paper, an AI-enabled intelligent defense framework for LFA in SDN networks is proposed to improve LFA detection accuracy and mitigation effect. Specifically, we present a Generative Adversarial Network (GAN) based LFA Detection (GLD) mechanism to obtain traffic patterns through adversarial training between generator and discriminator to improve detection accuracy. To enhance detection effectiveness, we design a data collection and processing mechanism to collect finer grained network status and switch status information in real time by leveraging in band network telemetry technology. Furthermore, we design a Double Deep Q-Network (DDQN) based LFA mitigation (DLM) mechanism, to effectively mitigate LFA traffic through the proposed rerouting algorithm and identify the suspicious IP addresses to block the attackers by using the proposed LFA source identification method. Simulation results demonstrate that compared with benchmark mechanisms, GLD mechanism can get higher detection accuracy and DLM mechanism can better mitigate LFA with smaller resource consumption cost.

**Index Terms**—LFA detection, LFA mitigation, GAN, DDQN, SDN.

Received 20 February 2025; revised 11 November 2025 and 17 March 2026; accepted 24 April 2026. Date of publication 28 April 2026; date of current version 11 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U22A2004 and Grant 62572106, in part by the Fundamental Research Funds for the Central Universities under Grant N25ZLV001, and in part by JSPS KAKENHI under Grant JP25K00373. Recommended for acceptance by M. Xu. (Corresponding author: Chuangchuang Zhang.)

Qiang He, Fuliang Li, and Xingwei Wang are with the College of Computer Science and Engineering, Northeastern University, Shenyang 110169, China.

Quanwei Li is with the College of Medicine and Biological Information Engineering, Northeastern University, Shenyang 110169, China.

Chuangchuang Zhang is with the College of Computer Science and Artificial Intelligence, Ludong University, Yantai 264025, China (e-mail: zhangcc@ldu.edu.cn).

Chi Xu is with the State Key Laboratory of Robotics, Shenyang Institute of Automation, Chinese Academy of Sciences, Shenyang 110016, China.

Ammar Hawbani is with the College of Computer Science and Engineering, Shenyang Aerospace University, Shenyang 110136, China.

Keping Yu is with the Graduate School of Science and Engineering, Hosei University, Tokyo 184-8584, Japan.

Digital Object Identifier 10.1109/TC.2026.3688596

## I. INTRODUCTION

As an emerging network architecture, Software Defined Network (SDN) can improve network flexibility and scalability, and achieve resource optimization by decoupling data plane from control plane [1]. In recent years, as the enabler for future networks, SDN has gained widespread popularity in academics and industries [2], [3]. Efficient services can be flexibly realized by leveraging logically centralized control feature of SDN. However, centralized control characteristics make SDN vulnerable to Distributed Denial of Service (DDoS) attacks [4]. In classic DDoS attacks, massive service requests are sent by bots to specific target servers continuously to cause the disruptions of legitimate services. According to 2023 Comcast Business Cybersecurity Threat Report released by Zayo in [5], DDoS attacks have become one of the most common network security incidents.

With the rapid development of network technologies, DDoS attacks have generated a variety of attack variants. As a new type of DDoS attacks, Link Flooding Attack (LFA) is more challenging in SDN networks. LFA aims to disconnect target links or network areas rather than specific servers by flooding legitimate traffic, thereby causing service degradation in target links or network areas [6]. LFA can not only launch attacks on three layers of SDN architecture, (i.e., data plane, control plane and application plane), but also cause significant damages to SDN networks, such as causing application crashes, disrupting normal SDN processes, and posing serious threats to network management and network security. It is difficult to distinguish LFA traffic from normal network traffic since LFA has small traffic and slow attack rate. These characteristics make it difficult to be detected and separate IP address of attack robots from normal network traffic. Thus, it is critical to accurately detect and effectively mitigate LFA in SDN networks.

To defense LFA attacks, many efforts have been devoted to developing numerous LFA detection or mitigation strategies in recent years [7]. Most existing defense solutions attempt to detect LFA traffic based on statistical analysis or machine learning. During LFA detection or mitigation process, controller and switches often frequently communicate. This not only occupies too many resources, but also has the limitations on data types and significant delays. It becomes more difficult to obtain network status information in real time, which is not conducive

to LFA detection. Moreover, traditional statistics based and machine learning based detection methods generally use relatively network features as a criterion to detect LFA. Due to the scarcity of datasets, these detection methods lack sufficient samples to perform statistical analysis or train machine learning models [8]. On the other hand, the majority of mitigation solutions try to reroute network flow based on rerouting strategy to mitigate the congestion of target links at significant resource costs. Thus, it is desirable to design an effective LFA defense mechanism to automatically detect and mitigate LFA at a small cost.

Traditional approaches often rely on fixed rules or signature-based detection, which are ineffective against evolving LFA patterns. The recent development of AI techniques provides a promising opportunity to overcome the problems of single features and insufficient training set [9]. Deep Neural Networks (DNNs) models have been used in DDoS attacks detection and mitigation since they can be trained by iterating on small batches of data without training on additional data [10]. Generative Adversarial Network (GAN) models are particularly suited for LFA detection and mitigation because they can learn subtle patterns in traffic data that traditional methods may miss. GANs create synthetic attack flows that mimic real LFA patterns, enabling the model to learn from a broader and more diverse dataset. The generator-discriminator competition forces the model to extract robust features that are resilient to noise and protocol variations, addressing the indistinguishable from normal traffic issue. By training on both real and synthetic data, GAN-based LFA detection approach can achieve better generalization across different network conditions. The advantages are not attainable with traditional congestion detection methods. Thus, GAN can address the lack problem of diverse labeled attack data in real-world scenarios by generating realistic attack traffic, which is a critical limitation in existing LFA detection methods.

In this paper, we design an AI-enabled intelligent LFA defense framework, to perform intelligent detection and mitigation of LFAs in SDN networks. Then, to accurately detect LFA, we design a GAN based LFA Detection (GLD) mechanism. Furthermore, to efficiently mitigate LFA, we design a Double Deep Q-Network (DDQN) based LFA Mitigation (DLM) mechanism. Our main contributions are as follows.

- We present LFAID, an AI-enabled LFA defense framework, to intelligently detect LFA with high accuracy, and quickly restore network traffic to mitigate LFA.
- We devise GLD mechanism, to improve LFA detection accuracy through adversarial training of generator and discriminator. To enhance detection effectiveness, we devise a In-band Network Telemetry (INT) based data collection algorithm to collect fine-grained network status and switch status information at a small cost.
- We devise DLM mechanism, to quickly mitigate congested links through DDQN based ReRouting (DRR) algorithm, and identify LFA attackers to mitigate LFA through LFA Source Identification (LSI) algorithm.
- We conduct simulation experiments for performance evaluation. Simulation results demonstrate compared with benchmark mechanisms, GLD mechanism can get higher

detection accuracy and DLM mechanism can mitigate LFA at a smaller resource cost.

The rest of this paper is organized as follows. Section II reviews related works. LFAID framework is shown in Section III. Section IV details GLD mechanism and DLM mechanism is presented in Section V. Section VI shows simulation experiments and this paper is concluded in Section VII.

## II. RELATED WORK

### A. LFA Detection

To defend LFA, Lu et al. [11] designed a service oriented Internet purification method by viewing malicious traffic filtering as a value-added service. To enhance filtering efficiency, a filter recommendation algorithm with minimum service cost and bandwidth damages is devised. Ravi et al. [12] designed a novel LFA detection and mitigation mechanism to detect and mitigate LFA via hybrid-SDN. The controller is used to collect link statistics information to detect LFA, and congested links are located to mitigate LFA. Aydeger et al. [13] devised a signaling game based stealthy LFA defense mechanism to mitigate stealthy LFAs by obtaining the optimal moving target defense strategy. Chen et al. [14] modeled the LFA defending problem as a two-person Bayesian game by considering LFA attacker and network manager, and achieved the optimal defense decision by deriving Bayesian Nash equilibrium. Wang et al. [15] used Renyi entropy and alarm threshold to determine the congested links and applied DRL to carry out traffic engineering to mitigate LFA traffic. Sun et al. [16] devised an improved weighed Euclidean distance mean shift algorithm to cluster network traffic based on delay feature and detect LFA based on clustering decision feature.

Hsieh et al. [17] designed a deep learning based detection framework to detect malicious LFA traffic and devised a convolutional neural network based rerouting algorithm to reduce detection time. Rasool et al. [18] designed a machine learning based security framework to detect LFA through machine learning repository to analyze real time network statistics. Li et al. [19] designed an adaptive LFA detection algorithm to detect malicious flows at line rate without pretrained models. Jiang et al. [20] recorded real time link and flow information via blockchain which enabled routers to trace the congested links. The LFA attacker is found based on similarity analysis. Cheng et al. [21] designed an extreme value based performance degradation model to detect LFA traffic by using the spatio-temporal graph convolutional network. Xie et al. [22] formulated the bandwidth resource allocation problem with LFA resilience as a multi-leader-multi-follower Stackelberg game model, and designed a differential pricing strategy to solve it.

### B. LFA Mitigation

Sun et al. [23] designed a low cost detection and mitigation system to detect LFA by introducing distributed relative entropy and mitigate LFA by using the rerouting algorithm. Rezapour et al. [24] monitored IP behaviors based on Dirichlet distribution and Bayesian statistics, and devised a Reinforcement

Learning (RL) based routing algorithm to mitigate LFA. Sun et al. [25] devised a lightweight defense framework to mitigate LFA at border switcher in a distribution manner, to reduce complexity and computational overhead. Li et al. [26] designed a temporal graph learning based early LFA intention prediction model to locate the LF. A distributed defense mechanism was devised to mitigate LFA. Jiang et al. [27] devised an in-network defense system to filter out suspicious LFA traffic through real time identification and rate limiting of LFA traffic, and mitigate LFA threats by using the programmable packet scheduling.

Ding et al. [28] designed an active link obfuscation algorithm to reroute the flow of target links to mislead LFA attacker, and extract unique traffic patterns of LFA based on support vector machine to block attack traffic. Murtuza et al. [29] designed a mitigation security framework to prevent LFA attacker from gathering network topology information. Huang et al. [30] designed a LFA defense method to proactively track and defense target links through adversary interest prediction, and devised a robust load balancing strategy to realize grouped online load balancing. Xie et al. [31] designed a SDN based lightweight load balancing algorithm to detect and defense against LFA by using random forest algorithm and gradient boosting decision tree algorithm. Wang et al. [32] designed a target link selection method and a congestion monitoring mechanism to detect LFA, and devised a multiple optional paths rerouting algorithm to mitigate the congested links. Murtuza et al. [33] investigated two link flooding based DDoS attacks including Coremelt and Crossfire, and created replicas of services under the attack to mitigate attacks.

Unlike previous works, we present an AI-enabled LFA detection and mitigation framework to perform intelligent detection and mitigation of LFAs. Different from most existing works merely collecting network flow information, we design a INT based information collection method to collect more types of data to enhance LFA detection effectiveness. Moreover, GLD mechanism does not require sufficient samples, and DLM mechanism can quickly mitigate LFA by blocking attack traffic of LFA bots at a small resource cost.

### III. AI-ENABLED INTELLIGENT DEFENSE FRAMEWORK

Fig. 1 depicts the proposed LFAID framework. Data plane consists of forwarding devices, such as switches. The core of LFAID framework consists of GLD module and DLM module, responsible for network monitoring, and LFA detection and mitigation. The core modules are as follows.

- **Network Monitoring:** It is responsible for real time monitoring of network status through SDN controller. Upon network status changing, controller will launch GLD and DLM modules to detect and mitigate LFA.
- **Data Collection:** It is responsible to collect fine grained network status and switch status information by employing INT technology.
- **Data Processing:** It is responsible to process collected data and extract data features from them. Data features and their labels are stored in the database.

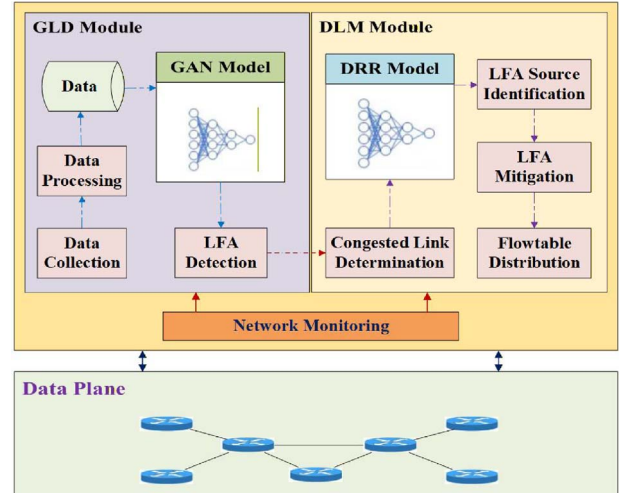


Fig. 1. The overall framework of LFAID.

- **GAN Model:** It is responsible to obtain LFA traffic patterns through adversarial training between generator and discriminator.
- **LFA Detection:** It is responsible to detect LFA based on GAN model.
- **Congested Link Determination:** It is responsible to determine congested target links caused by LFA.
- **DRR Model:** It is responsible to perform the rerouting of target links by using DDQN to alleviate congested links.
- **LFA Source Identification:** It is responsible to determine suspicious IP addresses issuing LFA traffic and block LFA traffic based on flow features and attacker behaviors.
- **LFA Mitigation:** It is responsible to mitigate LFA by using DRR algorithm and LSI method.
- **Flowtable Distribution:** After IP addresses of bots are identified, flowtables will be issued to discard packets from bots, thereby blocking LFA traffic.

The overall workflow of LFAID framework is comprised of LFA detection stage and LFA mitigation stage. In LFA detection stage, controller monitors network status in real time and collects network flow and switch status information by using INT technology. Then, feature extraction is performed including key flow feature and switch feature. We label each data record to train GAN model. Next, we apply GAN model to analyze network traffic and determine whether LFA has occurred. If there occurs no LFA, we will not take any actions, and network traffic will be forwarded normally.

In LFA mitigation stage, once LFA occurs, we perform the rerouting on congested links by executing DRR algorithm to alleviate LFA traffic. To further mitigate LFA, we trace LFA bots and record suspicious IP addresses based on flow features and attacker behaviors. The controller distributes flowtable rules to switches to discard packets sent by LFA bots. The controller re-assesses whether LFA persists. If LFA has been mitigated, the default routing strategy is restored promptly. Otherwise, the above LFA mitigation process continues until the LFA is completed mitigated.



**Algorithm 1:** Telemetry Data Insertion Frequency Adjustment.

---

**Input:** Insertion Frequency  $T$   
 Check Interval  $C$   
 Peak Threshold  $H$   
 Valley Threshold  $L$

**Output:** Adjusted Insertion Frequency  $T$

```

1: while Switch is monitoring do
2:   Get current timestamp  $t_c$ , last check time  $t_l$ , queue length  $Q$ ;
3:   if  $t_c - t_l > C$  then
4:      $t_l = t_c$ ;
5:     if  $Q > H$  then
6:        $T = T * 2$ ;
7:     else if  $Q < L$  then
8:        $T = T/2$ ;
9:     end if
10:  end if
11:  if  $t_c - t_l > T$  then
12:    Insert flag  $I = 1$ ;
13:  else if  $t_c - t_l < T$  then
14:    Insert flag  $I = 0$ ;
15:  end if
16: end while

```

---

## IV. GAN BASED LFA DETECTION MECHANISM

## A. Data Collection and Processing Method

1) *Data Collection*: Traditional data collection methods generally extract data packet information from switches, imposing excessive burden on switches and causing waste of system resources. The characteristics of SDN paradigm enable controller to collect network flow information with low overhead. The controller can send a Read-State Message request to switches through OpenFlow protocol to collect network flow information. We define the flow information collected at time slot  $t$  by  $A_t = (A_1, A_2, \dots, A_i, \dots)$ , where  $A_i$  is  $i$ -th network flow record. The features of each network flow can be denoted as a septuple  $A_i = (srP_i, dsP_i, ipP_i, rby_i, sby_i, rPk_i, sPk_i)$ , where  $srP_i$  is source IP address,  $dsP_i$  is destination IP address,  $ipP_i$  is IP protocol,  $rby_i$  is received bytes,  $sby_i$  is sent bytes,  $rPk_i$  is received packets,  $sPk_i$  is sent packets.

To improve detection accuracy, we devise a INT based switch status information collection method. INT can collect network status information by inserting metadata into network flows [34]. We define status information features (i.e., INT header information) of  $i$ -th switch at time slot  $t$  as an six-tuple  $B_i = (ID_i, Q_i, Ig_i, Eg_i, HL_i, BL_i)$ , where  $ID_i$  is switch ID,  $Q_i$  is switch queue length,  $Ig_i$  and  $Eg_i$  are the times when the packet enters and leaves the switch,  $HL_i$  is packet hop latency,  $BL_i$  is queue buffer latency.

INT uses data plane business traffic without control plane traffic to collect information. However, INT will occupy bandwidth resources, and need frequent insertion of telemetry information, thereby affecting switch forwarding efficiency. Thus, we design a telemetry data Insertion Frequency Adjustment (IFA) algorithm. As shown in Algorithm 1, we set up some status registers in switches to maintain queue length, check time intervals, and set a threshold to determine whether current network is busy. Switches dynamically adjust insertion frequency of INT header to control sample rate for telemetry information and reduce resource consumption.

2) *Data Processing*: The extracted features consist of qualitative features and quantitative features. For each qualitative

feature  $X$ , including  $srP_i$ ,  $dsP_i$ ,  $ipP_i$  and  $ID_i$ , we apply Shannon Entropy theorem [35] to calculate entropy value, to avoid model gradient explosion caused by over-fitting.

$$S(X) = \sum_{i=1}^N x_i \quad (1)$$

$$H(X) = -\frac{1}{S(X)} \sum_{i=1}^N x_i \log_2 \left( \frac{x_i}{S(X)} \right) \quad (2)$$

Where  $x_i$  is the number of occurrences of  $X$ ,  $S(X)$  is the number of times  $X$  appears.

For quantitative features including  $rby_i$ ,  $sby_i$ ,  $rPk_i$  and  $sPk_i$ , we execute the sum processing operation as follows.

$$by = \sum_{i=1}^N (rby_i + sby_i) \quad (3)$$

$$Pk = \sum_{i=1}^N (rPk_i + sPk_i) \quad (4)$$

For quantitative features including  $Q_i$ ,  $Ig_i$ ,  $Eg_i$ ,  $HL_i$  and  $BL_i$ , we calculate their average values respectively. After completing the above data process, we normalize all processed data to train GAN based detection model.

## B. GAN Based LFA Detection Algorithm

Traditional approaches often rely on fixed rules or signature-based detection, which are ineffective against evolving LFA patterns. By generating synthetic traffic with similar characteristics to real attacks, GLD algorithm enables proactive training of classifiers to detect LFA patterns. GANs are a class of generative frameworks that can learn the distribution of data and generate new sample data from the distribution [36]. A GAN model is comprised of generator and discriminator. Generator aims to generate fake sample data based on real data distribution, while discriminator aims to distinguish whether generated sample data is from real data distribution.

The GAN model does not directly use raw traffic data. Instead, it operates on pre-processed features extracted from network flow and switch status information since they are highly discriminative between normal traffic and LFA traffic. The real sample data in GLD contains both normal and LFA traffic. The distribution of generated sample data is defined by latent space of generator. Specifically, we sample from a multivariate Gaussian distribution with zero mean and identity covariance to generate random noise vectors. To ensure generated samples resemble real LFA traffic, we condition generator on attack-specific features. It allows the model to generate samples with similar properties to real LFA traffic.

Fig. 2 depicts GLD framework comprising of GAN model training and LFA detection. The random latent space provides input data to generator. Generator can generate fake sample data based on the distribution of input data, while discriminator can distinguish between real sample data and generated fake sample data. Therefore, we use generator to learn the distribution of real traffic including normal and LFA traffic, and apply discriminator to detect abnormal traffic.

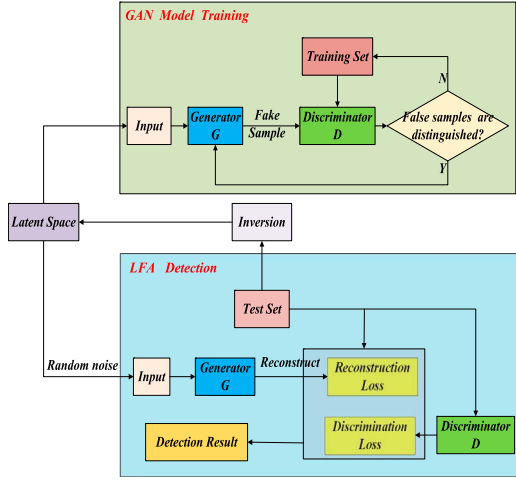


Fig. 2. The overall framework of GLD.

In GAN model training process, discriminator receives a mixed sample dataset, which is composed of fake sample data generated by generator and real sample data. Discriminator calculates evaluation values  $\logits_r$  and  $\logits_f$  of real sample data and generated fake sample data respectively. Discriminator aims to minimize the error between real sample data and generated fake sample data as follows.

$$Loss_{D_r} = \frac{1}{N} \sum_{i=1}^N |\logits_r - 1| \quad (5)$$

The goal of generator network is to minimize the error between generated fake sample data and its label as follows.

$$Loss_{D_f} = \frac{1}{N} \sum_{i=1}^N |\logits_f - 0| \quad (6)$$

Therefore, we can get the overall loss as follows.

$$Loss_D = Loss_{D_r} + Loss_{D_f} \quad (7)$$

As shown in Fig. 2, LFA detection is involved with discrimination stage and reconstruction stage. In discrimination stage, discriminator distinguishes between generated fake sample data and real sample data, severing as a direct anomaly detection tool. In reconstruction stage, generator generates sample data by performing the mapping from random noises in the latent space to real sample data space, i.e.,  $G_z: Z \rightarrow X$ .

By using the reverse mapping  $Z^g$ , we can calculate the residual  $R$  between generated data  $G(Z^g)$  and test data  $X_{test}$ , to quantify abnormal situation of data to detect potential LFA.

$$Loss_G = R(X_{test}) = \sum_{i=1}^N |X_{test,i} - G(Z^{g,i})| \quad (8)$$

The overall detection loss of GAN model can be as follows.

$$Loss_{GAN} = \lambda Loss_D + (1 - \lambda) Loss_G \quad (9)$$

The training process of discriminator can be regarded as a supervised logistic regression problem. The real sample data is labeled to 1 and generated sample data is labeled to 0. For each

sample  $X$ , we get the likelihood function of objective function of discriminator as follows.

$$l = [D_w(x)]^y [1 - D_w(x)]^{(1-y)} \quad (10)$$

Thus, the overall likelihood function can be expressed as

$$L = \prod_{i=1}^{n1} [D_w(x_{r,i})] \prod_{j=1}^{n2} [1 - D_w(x_{g,j})] \quad (11)$$

By taking the logarithm, we have:

$$L^* = \sum_{i=1}^{n1} \log [D_w(x_{r,i})] + \sum_{j=1}^{n2} \log [1 - D_w(x_{g,j})] \quad (12)$$

Discriminator aims to maximize the overall likelihood function, expressed as follows.

$$\max_w E_{x \sim p_{data}} \log [D_w(x)] + E_{x \sim p_g} \log [1 - D_w(x)] \quad (13)$$

During generator training, generator receives random noises extracted from latent space, and outputs generated sample data to discriminator. If generated sample data is judged as fake sample data, discriminator is penalized. Therefore, generator aims to maximize the output of discriminator.

$$\min_w E_{x \sim p_{data}} \log [D^*(x)] + E_{z \sim p_z} \log [1 - D^*(x)] \quad (14)$$

In the adversarial training process, generator  $G$  establishes the mapping between prior noises and real sample space, with data distribution  $P_z$ . To fool discriminator  $D$ , the distribution  $p_g$  of generated sample data should be similar as possible to the distribution  $p_{data}$  of real sample data. Thus, the objective function of generator  $G$  can be further expressed as follows.

$$\min \frac{1}{2} E_{z \sim p_z} \log (1 - D(G(z))) \quad (15)$$

Where  $D(G(z))$  is the predicted value of discriminator on the generated sample data.

Considering discriminator aims to classify sample accurately, the objective function can be further expressed as:

$$\max \frac{1}{2} E_{x \sim p_{data}} \log D(x) + \frac{1}{2} E_{z \sim p_z} \log (1 - D(G(z))) \quad (16)$$

The minimization game of generative adversarial network model is reflected in the maximum minimization conffliction between generator and discriminator, expressed as follows.

$$\min_G \max_D \frac{1}{2} E_{x \sim p_{data}} \log D(x) + \frac{1}{2} E_{z \sim p_z} \log (1 - D(G(z))) \quad (17)$$

The main workflow of GLD algorithm is shown in Algorithm 2. It takes training set, generator and discriminator as the input, and outputs trained generator and discriminator. In iterative training process,  $N$  real sample data is used to train and update model parameters. Generator is used to capture temporal features between samples, and discriminator is used to identify the classification of data to detect LFA anomalies.

**Algorithm 2:** GAN based LFA Detection.

---

**Input:** Generator  $G$   
 Discriminator  $D$   
 Training Set  $X$

**Output:** Trained Generator  $G$   
 Trained Discriminator  $D$

- 1: **while** Training not stop **do**
- 2:   Select  $N$  samples from  $X$  as real sample data;
- 3:   Generate  $N$  noise samples based on noise distribution  $P_z$ ;
- 4:   Generate  $N$  samples by  $G$ ;
- 5:   Use  $D$  to identify all real and generated samples;
- 6:   Calculate loss function of  $D$ ;
- 7:   Update parameters of  $D$  by gradient boosting algorithm;
- 8:   Generate  $N$  noise samples based on noise distribution  $P_z$ ;
- 9:   Calculate loss function of  $G$ ;
- 10:   Update parameters of  $G$  by using gradient boosting algorithm;
- 11: **end while**

---

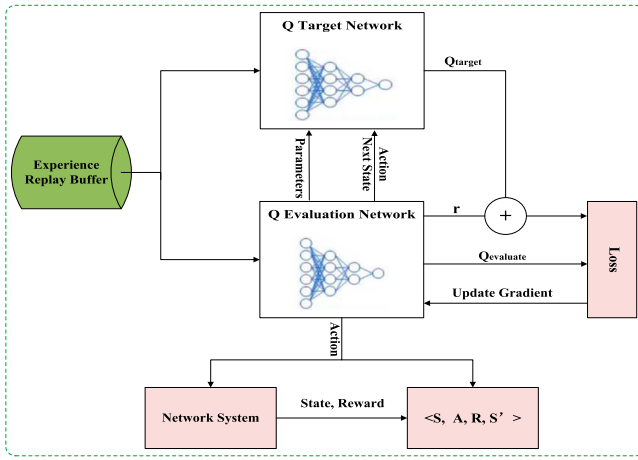


Fig. 3. The overall architecture of DDQN.

## V. DDQN BASED LFA MITIGATION

## A. DDQN Based ReRouting Algorithm

Once LFA is detected, DLM mechanism firstly determines the location of congested links, and records network flow of target links. DRR algorithm is executed to reroute network traffic on target links, to improve load balancing. Meanwhile, IP addresses of suspicious LFA attackers are determined. The corresponding flowtable rules are distributed by controller to switches to discard LFA traffic, thereby mitigating LFA.

Fig. 3 shows the overall architecture of DDQN. Evaluation network interacts with the environment to select actions based on current state. Target network is used to evaluate the Q-value of selected action. The parameters of target network will synchronize with those of evaluation network periodically. DDQN calculates rewards based on received network status and selected actions to update model parameters and select the best strategy. DDQN can adapt to dynamic network change through pre-trained model and low-frequency decision-making and online updates [37].

The state, reward, and action of DDQN are as follows.

**State:** To mitigate link congestion caused by LFA, we use bandwidth, latency, throughput, and link utilization of each network link as state space. The controller collects network status information through southbound interface.

**Reward:** To ensure service quality under the LFA, we use the average end-to-end delay ( $dly$ ) and packet loss rate ( $plr$ ) as the reward, expressed as follows.

$$r = \frac{1}{\omega_1 \cdot dly + \omega_2 \cdot plr} \quad (18)$$

$$dly = \frac{1}{n} \sum_{i=1}^n delay(f_i) \quad (19)$$

Where  $\omega_1 \in (0, 1)$  and  $\omega_2 \in (0, 1)$  are weight coefficients.  $f_i$  is the  $i$ -th data flow.

**Action:** DDQN learns the strategies to select actions based on status input to obtain higher reward. The action selection aims to optimize the weights of network links. Each network link is assigned to a basic weight, and routing decisions are made based on the weighted shortest path.

In DDQN, two neural networks are trained independently, one for selecting actions and the other for calculating the Q-value of actions. Evaluation network utilizes target network to calculate target Q-value, expressed as follows. Target network aims to provide stable Q-values.

$$Q_t = r + \gamma Q \left( s_{t+1}, \max_a Q(s_{t+1}, a; \theta^-); \theta^- \right) \quad (20)$$

Where  $\gamma$  is discount factor,  $\theta$  and  $\theta^-$  are the parameters of evaluation and target networks.

DDQN continuously updates the parameters through gradient descent algorithm to minimize loss function  $L(\theta)$ .

$$L(\theta) = E \left[ (Q_t - Q(s, a; \theta))^2 \right] \quad (21)$$

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta) \quad (22)$$

Where  $\nabla_{\theta} L(\theta)$  is the gradient of loss function.

The main workflow of DRR algorithm shown in Algorithm 3. DDQN uses an experience replay buffer to record historical states, actions and rewards. The parameters of DDQN model are updated to minimize loss function. Through continuous iteration and learning, the best actions in different states are selected to obtain higher rewards. Finally, a well trained DDQN model is determined.

## B. LFA Source Identification Algorithm

The rerouting of network traffic on target links under the LFA can only temporarily mitigate link congestion. LFA attacker can still send attack traffic to consume bandwidth resources of target links. Therefore, we further design a LFA source identification algorithm based on flow features and attacker behaviors. It can effectively identify IP addresses of robots. The data packets of LFA traffic will be discarded, thereby completely mitigating LFA. LSI algorithm consists of source identification based on flow features and source identification based on attacker behaviors.

1) *Flow Features Based LFA Source Identification:* LFA attacker operates botnets to send malicious traffic to flood target links. Once the LFA is detected, controller identifies affected network links. The related information of network flow passing through target links is recorded. Once network traffic on target

**Algorithm 3:** DDQN based ReRouting.

---

**Input:** Training Episode  $M$   
 Iteration Times  $T$   
 Update Interval  $U$   
**Output:** Trained DDQN model  $DN$

```

1: Initialize evaluation network  $Q_e$  with random weights  $\theta$ ;
2: Initialize target network  $Q_t$  with random weights  $\theta^- = \theta$ ;
3: Initialize experience replay buffer  $E = null$ ;
4: for  $i = 1 : M$  do
5:   Initialize state set  $S$ ;
6:   for  $t = 1 : T$  do
7:     Select action  $a$  with probability  $\varepsilon$ ;
8:     Execute action  $a$ ;
9:     Calculate reward  $r$  and next state  $s_{t+1}$ ;
10:     $e = (s_t, a, r, s_{t+1})$ ;
11:     $e \rightarrow E$ ;
12:    if  $E$  is large enough then
13:      Select  $K$  samples from  $E$  randomly;
14:      for  $j = 1 : K$  do
15:        Calculate  $Q_t$ ;
16:        Calculate  $Q(s_t, a; \theta)$ ;
17:        Update parameter  $\theta$ ;
18:      end for
19:    end if
20:    Update parameter  $\theta^- \leftarrow \theta$  every  $U$  steps;
21:     $s_t \leftarrow s_{t+1}$ ;
22:  end for
23: end for

```

---

links is rerouted, LFA attacker needs to re-launch the LFA. By extracting flow features, we can get source IP addresses of all network flows on target links. We record the set of all source IP addresses that appear on the attacked links, denoted by  $Y_1, Y_2, Y_3, \dots$ . As the number of attacks and rerouting increases, the probability of normal traffic appearing in the set will significantly decrease. Therefore, We can get suspicious LFA source IP addresses as follows.

$$Z = \cap_i^n Y_i \quad (23)$$

We mark suspicious IPs as key attention IPs. Then, we issue a random timed drop flowtable to prohibit network devices with these IP addresses from accessing the network for certain period. This will make LFA attacker believe zombie robots are no longer available. For each key attention IP, controller issues flowtable rules to block packets from that IP. As the number of LFA source identifications increases, LSI algorithm can effectively identify source IP addresses of LFA. The threshold in Eq. (23) can be determined based on a comprehensive analysis of both theoretical constraints and empirical observations. From a theoretical perspective, the threshold is closely tied to the trade-off between detection accuracy and false alarm rate. A higher threshold reduces the likelihood of false alarms. Conversely, a lower threshold enhances detection sensitivity but increases the risk of false alarms. To achieve a balance, extensive simulations are further required to determine the optimal threshold.

2) *Attacker Behaviors Based LFA Source Identification:* LFA attacker need get a network topology to control zombie robots to send many traceroute packets. After the LFA begins, attacker will control botnets to send malicious traffic to flood target links and continuously monitor target links to ensure the LFA is effective. After network traffic on target links is rerouted, LFA attacker must immediately rebuild network topology view and choose a new path to resend many traceroute packets in

a short time. This makes specific behavioral patterns of LFA attacker serve as key indicators to identify LFA source. In this work, we listen to traceroute packets after rerouting the traffic on congested links. We record hosts that sent traceroute packets during LFA interruption until the start of next LFA. By parsing source IP addresses of packets, we mark IP addresses of these hosts as suspicious IPs. If an IP address is marked multiple times, the flowtable rules are issued to block traffic from that IP. As the number of LFA rounds increases, most zombie robots can be successfully identified. Finally, the LFA can be completely mitigated.

## VI. SIMULATION EXPERIMENTS

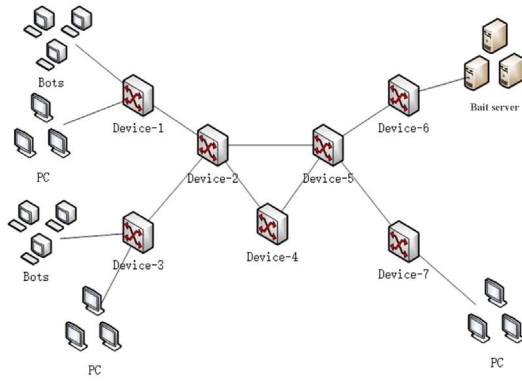
### A. Simulation Setups

The simulation experiment runs on a virtual machine on Ubuntu 20.04, with 10GB memory and 4 processor cores. As depicted in Fig. 4, similar to [30], we select two types of networks to simulate realistic SDN networks based on Mininet. The two networks both adopt a programmable data plane based on P4 and use ONOS as controller. The first network (*Network1*) is comprised of 7 switches, and several hosts and servers. Switches 1 and 3 are used as zombie robots to simulate LFA, and switch 6 is served as a bait. The second network (*Network2*) consists of 13 switches, and several hosts and servers. Switches 1, 4, and 12 are selected as zombie robots, and switch 11 is used as a decoy server. All PCs are set as the hosts to simulate normal user behaviors, and their locations are target area of LFA. we simulate LFAs by controlling botnets to send malicious traffic to bait servers near target areas.

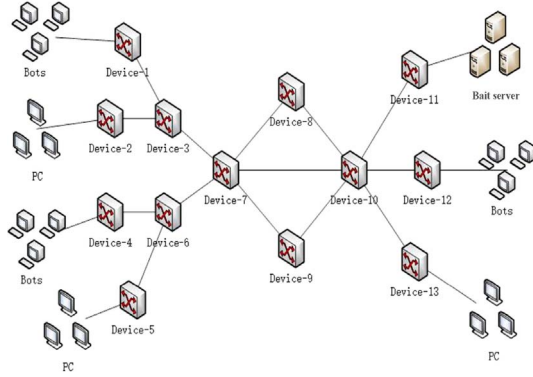
Iperf tool is used to simulate normal traffic and LFA traffic. The simulation time for data collection spans 4 hours, comprising 40 minutes of no-attack duration and 20 minutes of attack duration per hour, repeated across four cycles. Within no-attack period, 20 minutes are allocated to idle time and the other 20 minutes to network congestion time. We extract flow features from collected network flow information and switch status features, and normalize all processed data. We use Adam optimizer to train GAN model, and set learning rate is 0.0002 and batch size is 128. Generator adopts three layers of GRU units, while discriminator network is a four-layer fully connected network with LeakyReLU function. Noise dimensionality is 100.

To evaluate the performance of DRR algorithm, Shortest Path First (SPF) algorithm, Least Congestion Path (LCP) algorithm and SDN's own Rerouting (SDNR) algorithm are selected as comparison algorithms. We find that when the detection threshold of source IP is 5, detection accuracy is the highest through multiple experiments. Therefore, we set detection threshold is 5, to avoid missed detections caused by excessively high thresholds or false alarms caused by overly low thresholds. Recurrent Neural Networks based Detection (RNND) mechanism [38], LSTM-based collaborative Detection (LSTMD) mechanism [39] and Gated Recurrent Units (GRU) based Detection (GRUD) mechanism [40] are selected as comparison mechanisms of GLD mechanism. To evaluate DLM mechanism, three mitigation mechanisms including LFA mitigation





(a) Network1



(b) Network2

Fig. 4. Network Topology.

(NLM) mechanism without taking any mitigation actions, optimized Rerouting based LFA Mitigation (RLM) mechanism [23] and Lightweight LFA Mitigation (LLM) mechanism [25] are selected in the simulations.

### B. Simulation Results

We evaluate the performance of GLD mechanism based on *Network1*, and evaluate the performance of DLM mechanism based on two topologies.

1) *GAN Based LFA Detection Mechanism Evaluation:* As depicted in Fig. 5, the collected data is divided into training set and test set, with the ratio about 3:1. Fig. 6 depicts traffic changes of network *Network1* under normal and LFA state. For a long time period after the LFA occurs, the status of network traffic does not undergo significant changes. This is because IFA algorithm can reduce data collection frequency during network congestion.

Fig. 7 depicts loss variation of generator and discriminator under different batch sizes. In the early stage, generator cannot generate sample data that is close to real sample data, while discriminator can easily distinguish between real data and generated data. This results in a higher loss for generator and a lower loss for discriminator. As the training epochs increase, generator gradually learns how to generate realistic data. The loss of generator gradually decreases. When batch size is 128,

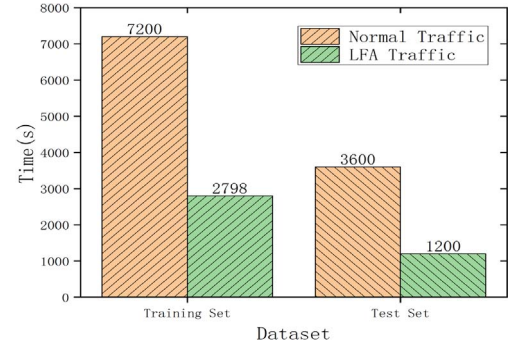


Fig. 5. Dataset.

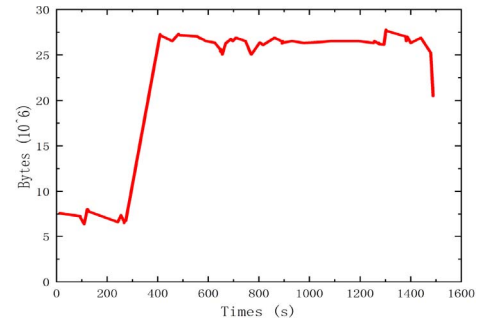


Fig. 6. Variations in network traffic.

the loss of generator and discriminator both gradually converges to stable values.

Fig. 8 depicts confusion matrix comparison of four detection mechanisms. GLD mechanism outperforms comparison mechanisms on reducing false positives and false negatives. Fig. 9 depicts performance comparison of four detection mechanisms. Compared with benchmark mechanisms, GLD mechanism can achieve higher accuracy, recall and F1 Score.

2) *DDQN Based LFA Mitigation Mechanism Evaluation:* We evaluate the effectiveness and efficiency of DRR algorithm. Fig. 10 depicts network delay of four routing algorithms in *network2*. It can be observed that DRR algorithm can get smaller network delay than SDNR, SPF and LCP algorithms. Fig. 11 depicts network delay of four mitigation mechanisms. The delay of both networks is close to zero before the LFA begins. After the LFA starts, network delay quickly increases. NLM mechanism maintains a high delay. This is because it takes no LFA mitigation action. LFA attacker successfully congests target links with just one round of LFA. DLM mechanism can more effectively reduce network delay than comparison mechanisms. This is because DRR algorithm distributes traffic on target links to other links in the early stage of LFA, thereby mitigating link congestion. After three rounds of LFAs, most of LFA sources are successfully identified. There are not enough zombie robots to re-launch effective LFA.

Fig. 12 depicts packet loss rate in both networks. DLM mechanism can get lower packet loss rate than comparison mechanisms. Since no mitigation measures are taken, packet loss rate of NLM mechanism rises. This is because different



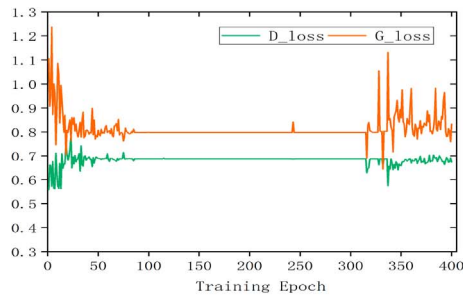
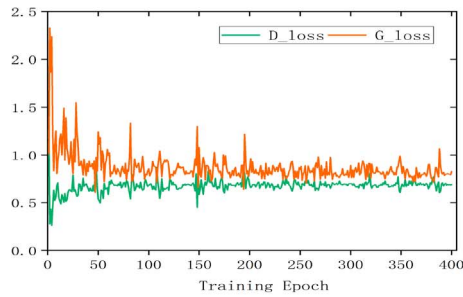
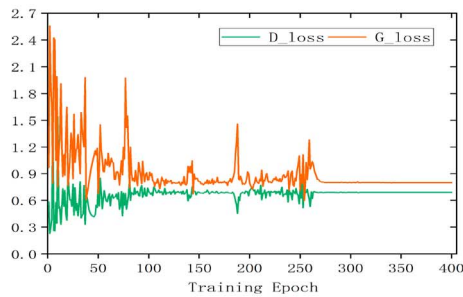
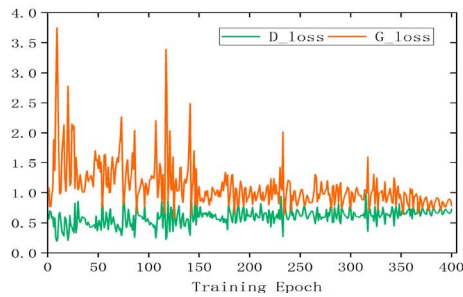
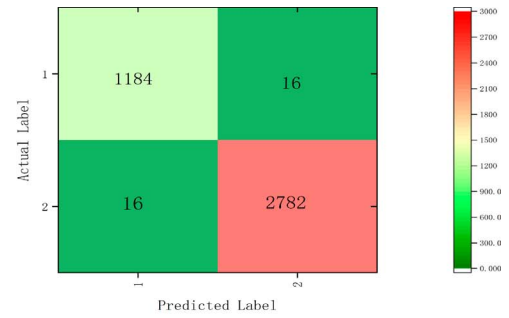
(a) *Batch\_size* = 32(b) *Batch\_size* = 64(c) *Batch\_size* = 128(d) *Batch\_size* = 256

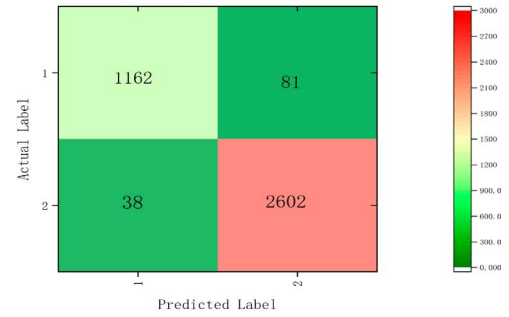
Fig. 7. Loss variations of generator and discriminator.

from comparison mechanisms, DLM mechanism effectively mitigates LFA by rerouting the congested links and blocking attack sources based on flow features and attacker behaviors.

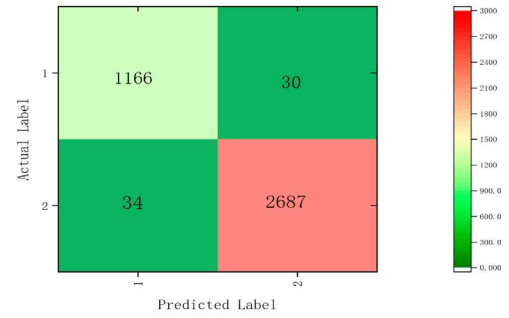
Fig. 13 depicts network jitter change. DLM mechanism has stronger network stability than comparison mechanisms. When both networks do not perform any operations, network jitter increases and shows significant fluctuations. It means the stability of network performance significantly decreases. When we apply DLM mechanism to both networks, the amplitude and frequency of network jitter changes are significantly reduced, and peak value is reduced.



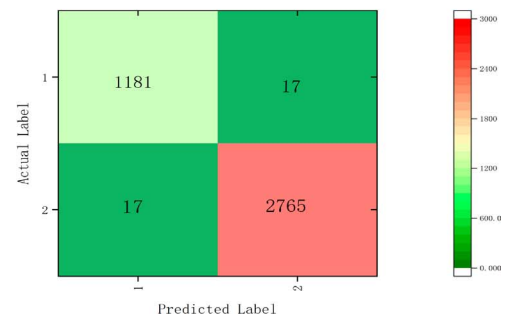
(a) GLD



(b) RNND



(c) GRUD



(d) LSTMD

Fig. 8. Confusion matrix.

Fig. 14 depicts source identification results of DLM mechanism under three rounds of LFAs. DLM mechanism can get a relatively high identification success rate in both networks. Moreover, the number of successfully identified robots increases as LFA rounds increases. DLM mechanism makes controller issue flowtables to block malicious traffic sent by zombie robots.

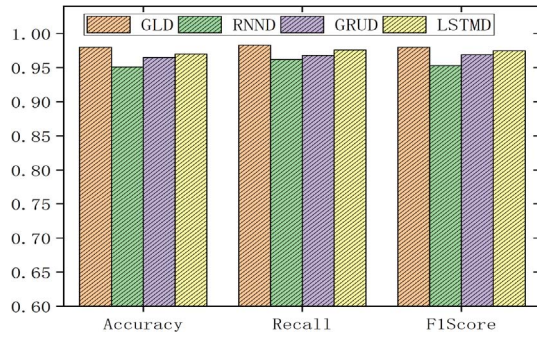


Fig. 9. LFA detection results.

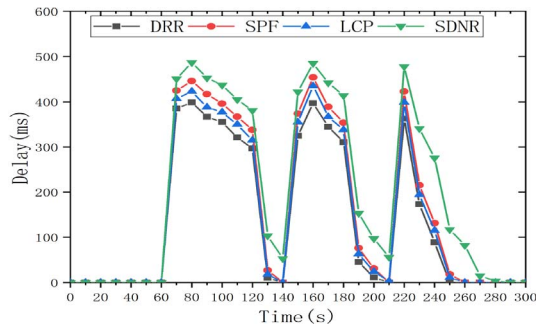
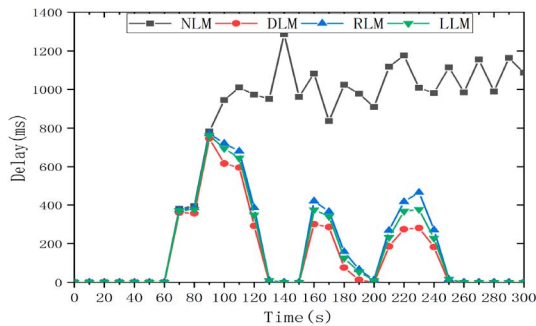
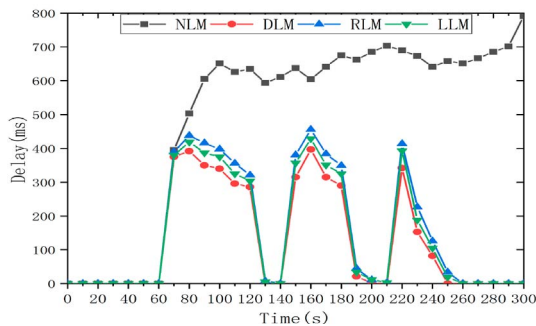


Fig. 10. Delay comparison.



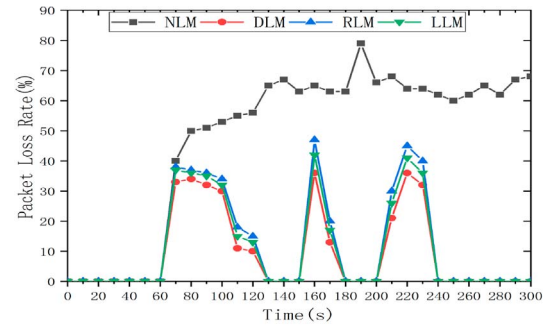
(a) Network1



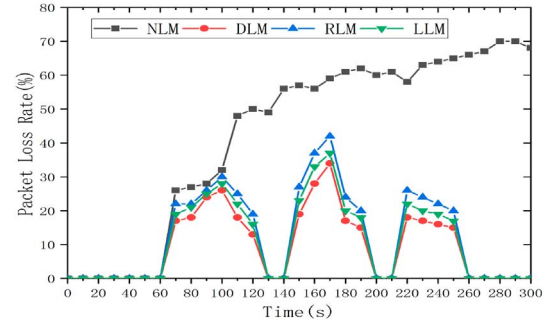
(b) Network2

Fig. 11. Network delay.

Fig. 15 depicts resources consumption comparison. DLM mechanism consumes the smallest CPU resources and memory resources. This is because considering LFA persistence, DLM

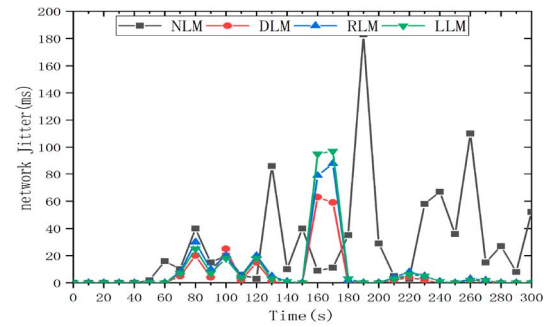


(a) Network1

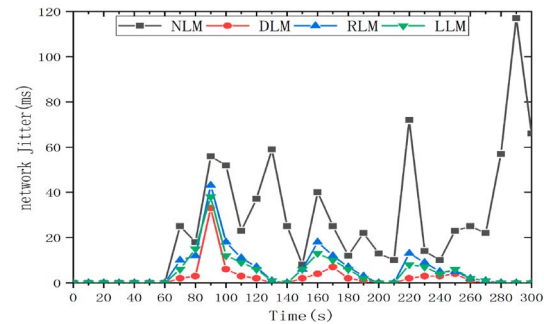


(b) Network2

Fig. 12. Packet loss rate.



(a) Network1



(b) Network2

Fig. 13. Network jitter.

mechanism only need make corresponding routing adjustment decisions after detecting the LFA. DRR algorithm which uses pre-trained and low-frequency decision based on DDQN, can maintain real-time and stability requirements of model.

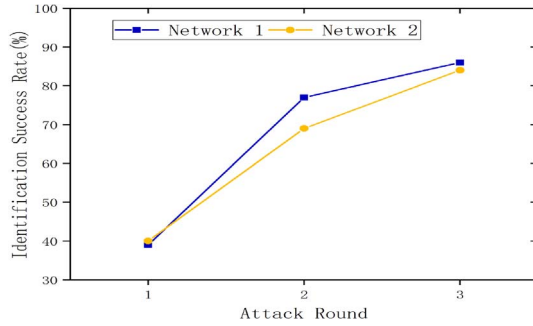


Fig. 14. LFA source identification.

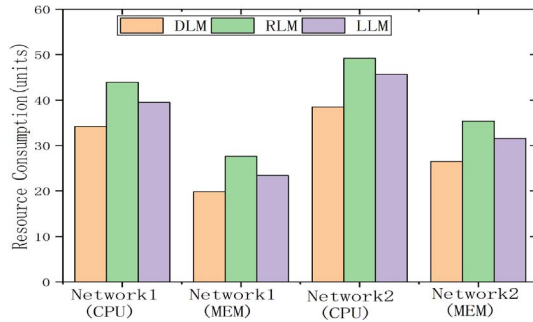


Fig. 15. Resource consumption.

### C. Discussion

The above simulation results demonstrate the effectiveness and efficiency of our proposed detection and migration mechanisms. With the expansion of network scale, more network and switch status information will be collected. It need take more time to train GAN model, resulting in high cost of LFA detection in large scale networks. On the other hand, LFA source identification becomes more complex in large scale networks due to a large amount of network devices. How to effectively detect and mitigate LFAs in large scale networks becomes very challenging. Therefore, we plan to extend our performance evaluations to larger scale networks in future work, which will involve optimizing the computational efficiency of GAN and DDQN. Additionally, we will explore distributed training strategies to enhance scalability.

Although SDN has an inherent ability to automatically detect congestion and alleviate it through rerouting, LFA defense research remains indispensable for several critical reasons. Firstly, the native congestion handling capability of SDN is limited. SDN's inherent rerouting mechanisms primarily rely on reactive strategy triggered after network congestion or link failures occur. However, this reactive strategy introduces inevitable latency between the onset of congestion and the implementation of a solution, resulting in data loss, service degradation, or operational failures. By contrast, our proposed solution adopts a proactive method to monitor link congestion and enable preemptive rerouting. Secondly, SDN is difficult to effectively respond to LFA. Malicious attacks including LFA

can exploit SDN's centralized control plane to launch denial-of-service attacks by intentionally congesting critical links. SDN's native mechanisms may not distinguish between legitimate congestion and malicious traffic, leading to inefficient or incorrect rerouting decisions. Our proposed LFA detection and mitigation mechanism can efficiently identify abnormal traffic patterns associated with attacks, and mitigate LFA by using the proposed mitigation strategy.

## VII. CONCLUSION

In this paper, we address the LFA detection and mitigation problem in SDN networks by proposing an intelligent LFA defense framework. To enhance detection efficiency and accuracy, we design a GLD mechanism to automatically detect LFA traffic and a data collection and processing method to collect finer grained network status and switch status information. Furthermore, we design a DLM mechanism to mitigate LFA traffic by using DDQN based rerouting algorithm to improve load balancing and block attackers by identify suspicious IP addresses. Final simulation results demonstrate the effectiveness and efficiency of the proposed LFA defense mechanisms. In future work, we will investigate the LFA defense problem in dynamic large-scale SDN networks where network status is dynamic and uncertain.

## REFERENCES

- [1] Y. Yue et al., "DeepSelector: A deep learning-based virtual network function placement approach in SDN/NFV-enabled networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1759–1773, Mar. 2025.
- [2] Q. He et al., "Routing optimization with deep reinforcement learning in knowledge defined networking," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1444–1455, Feb. 2024.
- [3] C. Zhang et al., "Intelligent task offloading and resource allocation in knowledge defined edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 5, pp. 4312–4325, May 2025.
- [4] T. Cai et al., "ADAM: An adaptive DDoS attack mitigation scheme in software-defined cyber-physical system," *IEEE Trans. Ind. Informat.*, vol. 19, no. 6, pp. 7802–7813, Jun. 2023.
- [5] "2023 comcast business cybersecurity threat report," Comcast, 2023. [Online] Available: <https://business.comcast.com/community/browse-all/details/2023-comcast-business-cybersecurity-threat-report>
- [6] R. Ur Rasool et al., "A survey of link flooding attacks in software defined network ecosystems," *J. Netw. Comput. Appl.*, vol. 172, Dec. 2020, Art. no. 102803.
- [7] S. Kautish, A. Reyana, and A. Vidyarthi, "SDMTA: Attack detection and mitigation mechanism for DDoS vulnerabilities in hybrid cloud environment," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6455–6463, Sep. 2022.
- [8] H. Luo, Z. Liu, and S. Zhang, "Preventing DDoS flooding attacks with cryptographic path identifiers in future internet," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 1690–1704, Jun. 2022.
- [9] Q. He et al., "A blockchain-based scheme for secure data offloading in healthcare with deep reinforcement learning," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 65–80, Feb. 2024.
- [10] V. Nhamte et al., "DDoS attack detection and mitigation using deep neural network in SDN environment," *Comput. Secur.*, vol. 138, Mar. 2024, Art. no. 103661.
- [11] N. Lu et al., "STOP: A service oriented internet purification against link flooding attacks," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 938–953, 2022.
- [12] N. Ravi, S. M. Shalinie, and D. D. J. Theres, "Balance: Link flooding attack detection and mitigation via hybrid-SDN," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 3, pp. 1715–1729, Sep. 2020.
- [13] A. Aydeger et al., "Strategic defense against stealthy link flooding attacks: A signaling game approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 1, pp. 751–764, Jan.–Mar. 2021.



- [14] X. Chen et al., "Defending against link flooding attacks in Internet of Things: A Bayesian game approach," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 117–128, Jan. 2022.
- [15] J. Wang et al., "ReLFA: Resist link flooding attacks via Renyi entropy and deep reinforcement learning in SDN-IoT," *China Commun.*, vol. 19, no. 7, pp. 157–171, 2022.
- [16] W. Sun and C. Wang, "An improved mean shift clustering algorithm for LFA detection," in *Proc. IEEE 21st Int. Conf. Softw. Qual., Rel. Secur. Companion (QRS-C)*, 2021, pp. 926–934.
- [17] C. H. Hsieh et al., "Efficient detection of link-flooding attacks with deep learning," *Sustainability*, vol. 13, no. 22, 2021, Art. no. 12514.
- [18] R. U. Rasool et al., "CyberPulse++: A machine learning-based security framework for detecting link flooding attacks in software defined networks," *Int. J. Intell. Syst.*, vol. 36, no. 8, pp. 3852–3879, 2021.
- [19] Y. Li et al., "In-network collaborative link flooding attack defense with adaptive anomaly analysis," in *Proc. IEEE Global Commun. Conf.*, 2023, pp. 1717–1722.
- [20] X. Jiang et al., "Credible link flooding attack detection and mitigation: A blockchain-based approach," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3537–3554, Jun. 2024.
- [21] S. Cheng et al., "LinkGuard: Link flooding attack detection and mitigation via spatio-temporal graph convolutional network," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4059–4075, Sep./Oct. 2024.
- [22] L. Xie et al., "Differential pricing strategies for bandwidth allocation with LFA resilience: A Stackelberg game approach," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4899–4914, 2023.
- [23] H. Sun et al., "A low-cost detection and mitigation method for link flooding attacks in SDN," in *Proc. IEEE 23rd Int. Conf. Commun. Technol. (ICCT)*, 2023, pp. 1225–1230.
- [24] A. Rezapour and W. G. Tzeng, "RL-shield: Mitigating target link-flooding attacks using SDN and deep reinforcement learning routing algorithm," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 4052–4067, Nov./Dec. 2022.
- [25] H. Sun et al., "Mitigating while accessing: A lightweight defense framework against link flooding attacks in SDN," *China Commun.*, vol. 21, no. 11, pp. 15–27, 2024.
- [26] Y. Li et al., "A collaborative programmable LFA defense using temporal graph learning in AIoT," *IEEE Internet Things J.*, vol. 12, no. 3, pp. 2514–2529, Feb. 2025.
- [27] W. Jiang et al., "SatShield: In-network mitigation of link flooding attacks for LEO constellation networks," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 27340–27355, Aug. 2024.
- [28] X. Ding et al., "Active link obfuscation to thwart link-flooding attacks for Internet of Things," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, 2020, pp. 217–224.
- [29] S. Murtuza and K. Asawa, "Early prevention and mitigation of link flooding attacks in software defined networks," *J. Netw. Comput. Appl.*, vol. 224, Apr. 2024, Art. no. 103832.
- [30] Z. Huang et al., "Defending against link-flooding attacks with adversary interest prediction and grouped online load balancing," *IEEE Trans. Netw.*, vol. 33, no. 3, pp. 1056–1069, Jun. 2025.
- [31] J. Xie et al., "LFA-Cleaner: mitigating link flooding attacks in software-defined networks," in *Proc. IEEE Int. Conf. Mobile Internet, Cloud Comput. Inf. Secur. (MICCIS)*, 2023, pp. 154–161.
- [32] J. Wang et al., "Detecting and mitigating target link-flooding attacks using SDN," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 6, pp. 944–956, Nov./Dec. 2019.
- [33] S. Murtuza and K. Asawa, "Detection and mitigation of link flooding-based DDoS attacks on a software defined network using network function virtualisation," *Int. J. Commun. Netw. Distrib. Syst.*, vol. 30, no. 2, pp. 202–226, 2024.
- [34] P. Zhang et al., "AdapINT: A flexible and adaptive in-band network telemetry system based on deep reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 5, pp. 5505–5520, 2024, Oct.
- [35] L. D. Tsobdjou, S. Pierre, and A. Quintero, "An online entropy-based DDoS flooding attack detection system with dynamic threshold," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 1679–1689, Jun. 2022.
- [36] E. Brophy et al., "Generative adversarial networks in time series: A systematic literature review," *ACM Comput. Surv.*, vol. 55, no. 10, pp. 1–31, 2023.
- [37] Z. Li, Y. Kong, and C. Jiang, "A transfer double deep Q network based DDoS detection method for Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 5317–5331, Apr. 2023.
- [38] H. Polat et al., "A novel approach for accurate detection of the DDoS attacks in SDN-based SCADA systems based on deep recurrent neural networks," *Expert Syst. Appl.*, vol. 197, Jul. 2022, Art. no. 116748.
- [39] S. Yeom, C. Choi, and K. Kim, "LSTM-based collaborative source-side DDoS attack detection," *IEEE Access*, vol. 10, pp. 44033–44045, 2022.
- [40] S. Ur Rehman et al., "DIDDOS: An approach for detection and identification of distributed denial of service (DDoS) cyberattacks using gated recurrent units (GRU)," *Future Gener. Comput. Syst.*, vol. 118, pp. 453–466, May 2021.



**Qiang He** (Associate Member, IEEE) received the Ph.D. degree in computer application technology from the Northeastern University, Shenyang, China, in 2020. He also worked as a Visiting Ph.D. Researcher with the School of Computer Science and Technology, Nanyang Technical University, Singapore, from 2018 to 2019. He is currently a Professor with the College of Computer Science and Engineering, Northeastern University. His research interests include machine learning, social network analytic, data mining, and health care. He has published more than 70 journal articles and conference papers.



**Quanwei Li** received the B.S. degree from Anhui University of Chinese Medicine, China. He is currently working toward the M.S. degree with the Northeastern University in Shenyang, China. His research interest includes edge computing.



**Chuangchuang Zhang** received the Ph.D. degree in computer application technology from the Northeastern University, Shenyang, China, in 2020. He is currently an Associate Professor with the College of Computer Science and Artificial Intelligence, Ludong University, China. He has published more than 40 Journal/conference papers. His research interests include, service function chain orchestration, Internet of Things, and mobile edge computing.



**Fuliang Li** (Member, IEEE) received the B.S. degree from the Northeastern University, China, in 2009, and the Ph.D. degree from Tsinghua University, China, in 2015, both in computer science. He is currently a Professor with the College of Computer Science and Engineering, Northeastern University. He was a Postdoctoral Fellow with the Department of Computing, Hong Kong Polytechnic University, Hong Kong, from 2016 to 2017. He has published 50 Journal/conference papers, including journal papers. His research interests include network management and measurement, mobile computing, and software-defined networking.



**Xingwei Wang** received the B.S., M.S., and Ph.D. degrees in computer science from the Northeastern University, Shenyang, China, in 1989, 1992, and 1998, respectively. He is currently a Professor with the College of Computer Science and Engineering, Northeastern University. He has published more than 100 journal articles and conference papers. His research interests include cloud computing and future internet.





industrial control systems, industrial wireless networks, edge computing, and artificial intelligence.

**Chi Xu** (Senior Member, IEEE) received the Ph.D. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2017. He is currently a Professor with Shenyang Institute of Automation, Chinese Academy of Sciences, Shenyang, China. He is a voting member of IEEE 1918.1 working group for tactile internet as well as a member of IEEE 1932.1 working group for licensed/unlicensed spectrum interoperability in wireless mobile networks. He also serves as a standardization delegate for 3GPP TSG RAN. His research interests include



**Ammar Hawbani** received the B.S., M.S., and Ph.D. degrees in computer software and theory from the University of Science and Technology of China (USTC), Hefei, China, in 2009, 2012, and 2016, respectively. He is an Associate Professor of networking and communication algorithms with the School of Computer Science and Technology, USTC. From 2016 to 2019, he worked as a Post-doctoral Researcher with the School of Computer Science and Technology, USTC.



intelligence, blockchain, and information security.

**Keping Yu** (Senior Member, IEEE) received the M.E. and Ph.D. degrees from the Graduate School of Global Information and Telecommunication Studies, Waseda University, Japan, in 2012 and 2016, respectively. He is currently an Associate Professor, the Vice Director of the Institute of Integrated Science and Technology, and the Director of the Network Intelligence and Security Laboratory (YU Lab), Hosei University, Japan. His research interests include smart grids, information-centric networking, the Internet of Things, artificial intelligence, blockchain, and information security.